

软件过程改进方案设计文档

智能自动贩卖机项目

前提：本方案以前期《基于ISO/IEC 12207/15504/33001的自动贩卖机项目软件过程定义》为基础，结合《AI技术应用场景匹配报告》识别出的5个关键场景，设计AI技术与传统软件过程的融合改进方案。

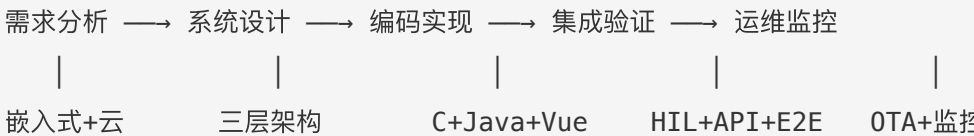
一、过程重构总体思路

1.1 设计原则

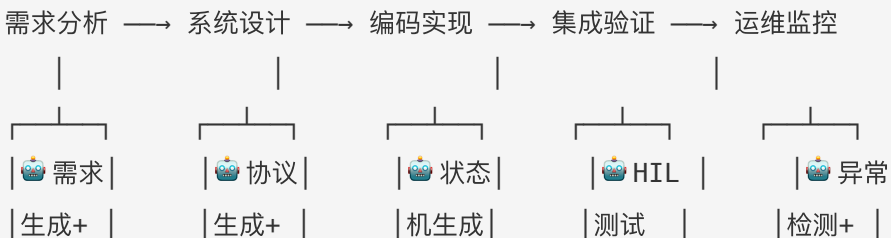
- 渐进式融入：**AI技术以增量方式引入，从编码和协议检查开始，逐步扩展到测试和运维，避免对嵌入式开发团队的冲击
- 人机协同：**AI定位为"增强智能"，关键安全决策（支付、固件烧录、OTA升级）保留人工审批
- 硬件适配：**嵌入式场景中AI的适用性需要考虑硬件约束（MCU资源有限、实时性要求），AI辅助集中在代码生成和审查环节，不引入运行时的AI依赖
- 跨栈一致性：**利用AI强化嵌入式固件、云平台API、移动端之间的接口一致性检查

1.2 改进概览

传统贩卖机SDLC流程（ISO/IEC 12207裁剪后）：



AI增强后流程（新增AI节点以🤖标注）：



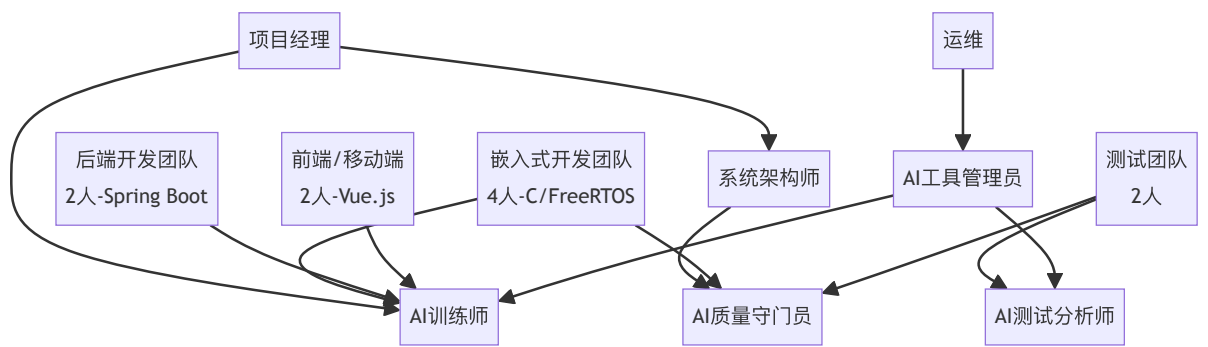
合规	架构	+审查	生成	根因
检查	推荐	+重构	+自愈	分析

二、新增AI相关角色定义

2.1 角色总览

角色名称	职责概述	与贩卖机项目的特殊适配
AI训练师	管理Prompt工程、构建嵌入式领域知识库、培训团队AI使用	需建立嵌入式C/FreeRTOS/状态机/MQTT协议的专用知识库
AI测试分析师	管理AI测试工具、评估AI生成的测试用例质量、设计HIL测试策略	需理解硬件在环测试的物理约束，判断AI生成的异常场景的物理可行性
AI质量守门员	审查AI生成产出的质量、确保嵌入式代码安全	重点审查中断处理、内存操作、外设寄存器访问等嵌入式高风险代码
AI工具管理员	选型、部署、维护AI工具链、管理硬件仿真环境与AI工具的集成	需在硬件仿真环境和云开发环境之间统一AI工具链

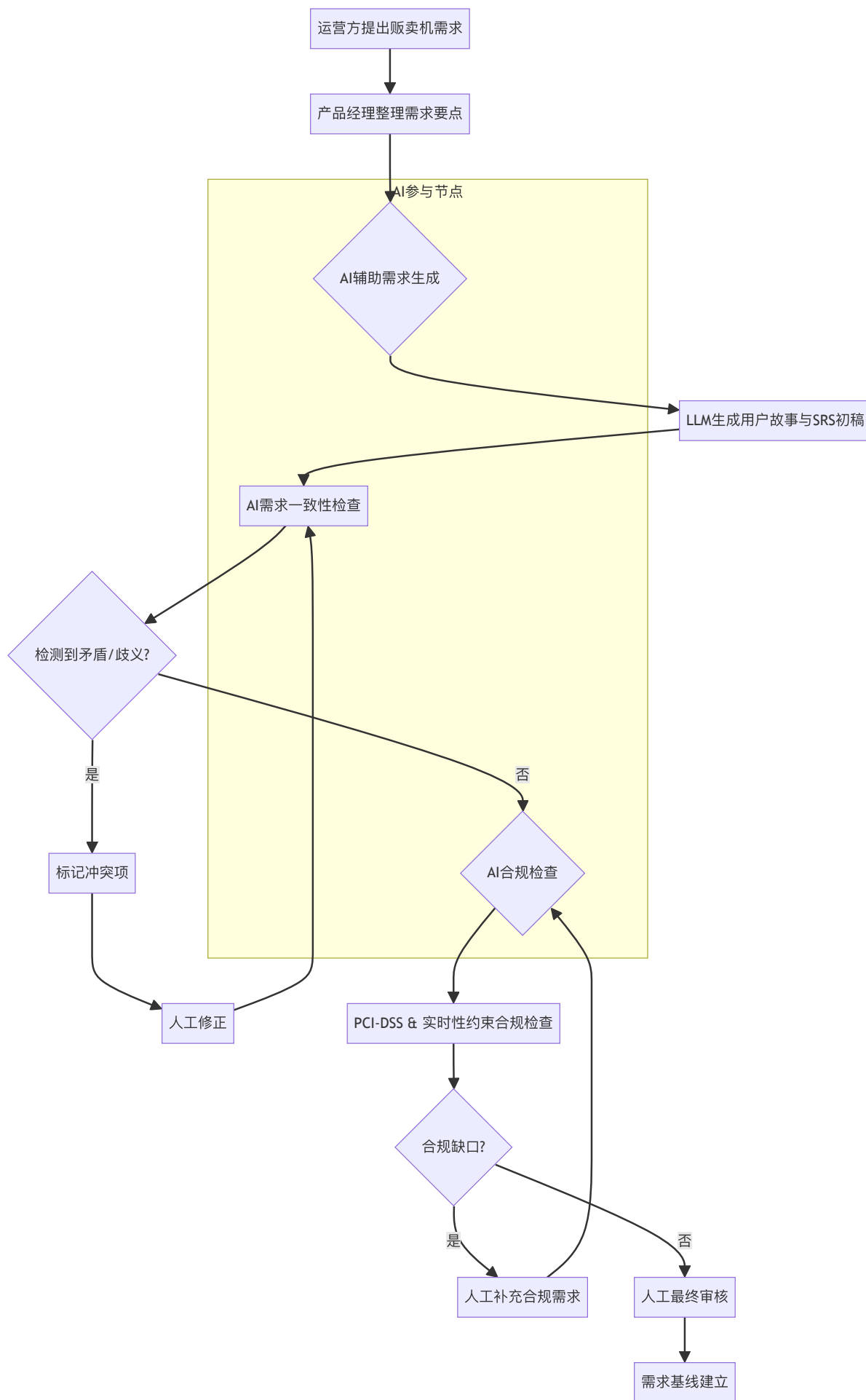
2.2 与传统角色关系



三、各阶段流程改进方案

3.1 需求分析阶段改进

改进后流程图

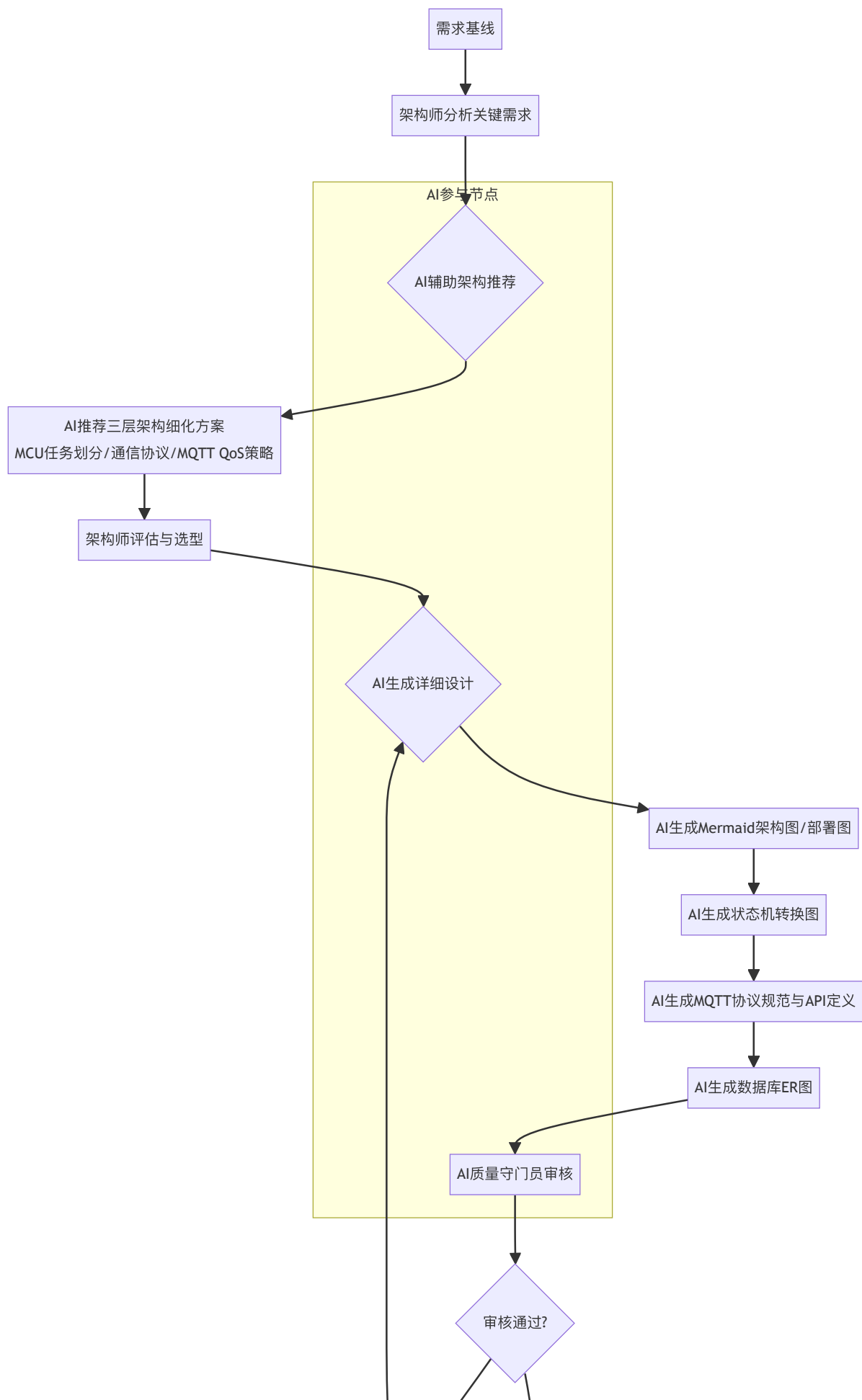


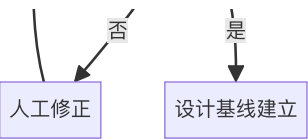
改进点说明

改进项	传统方式	AI增强方式	预期效果
SRS编写	产品经理手工编写	LLM基于要点生成初稿，人工修订	编写时间减少50%
需求一致性	人工评审偶然发现	AI语义分析自动检测（如库存预警与实时响应之间的潜在冲突）	需求缺陷减少30%
合规检查	安全专家逐条对照PCI-DSS	AI自动检查合规清单，标记缺失项	合规遗漏减少35%

3.2 系统设计阶段改进

改进后流程图



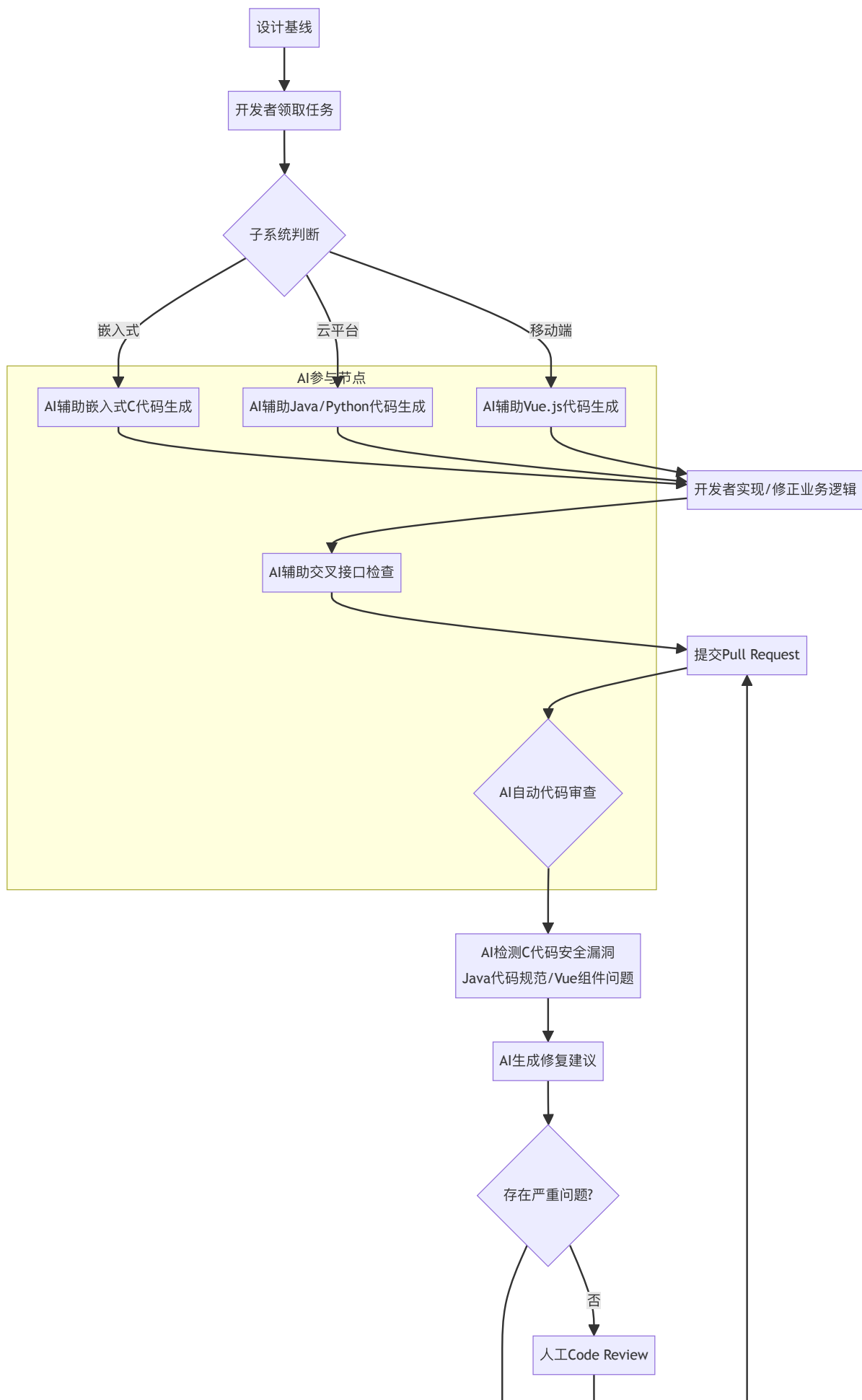


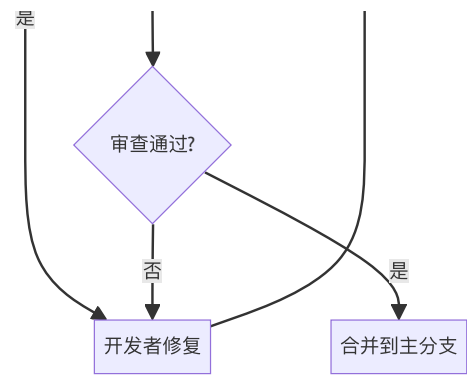
改进点说明

改进项	传统方式	AI增强方式	预期效果
架构细化	架构师手工调研与设计	AI基于需求推荐MCU任务优先级划分、通信QoS策略等	架构设计时间减少35%
状态机设计	Visio手工绘制	AI基于状态转换规则生成Mermaid状态图	状态机设计效率提升60%
通信协议定义	手工编写MQTT Topic和Payload格式文档	AI自动生成结构化的协议规范，并检查嵌入式与云端定义一致性	协议文档编写时间减少55%，不一致缺陷减少40%

3.3 编码实现阶段改进

改进后流程图



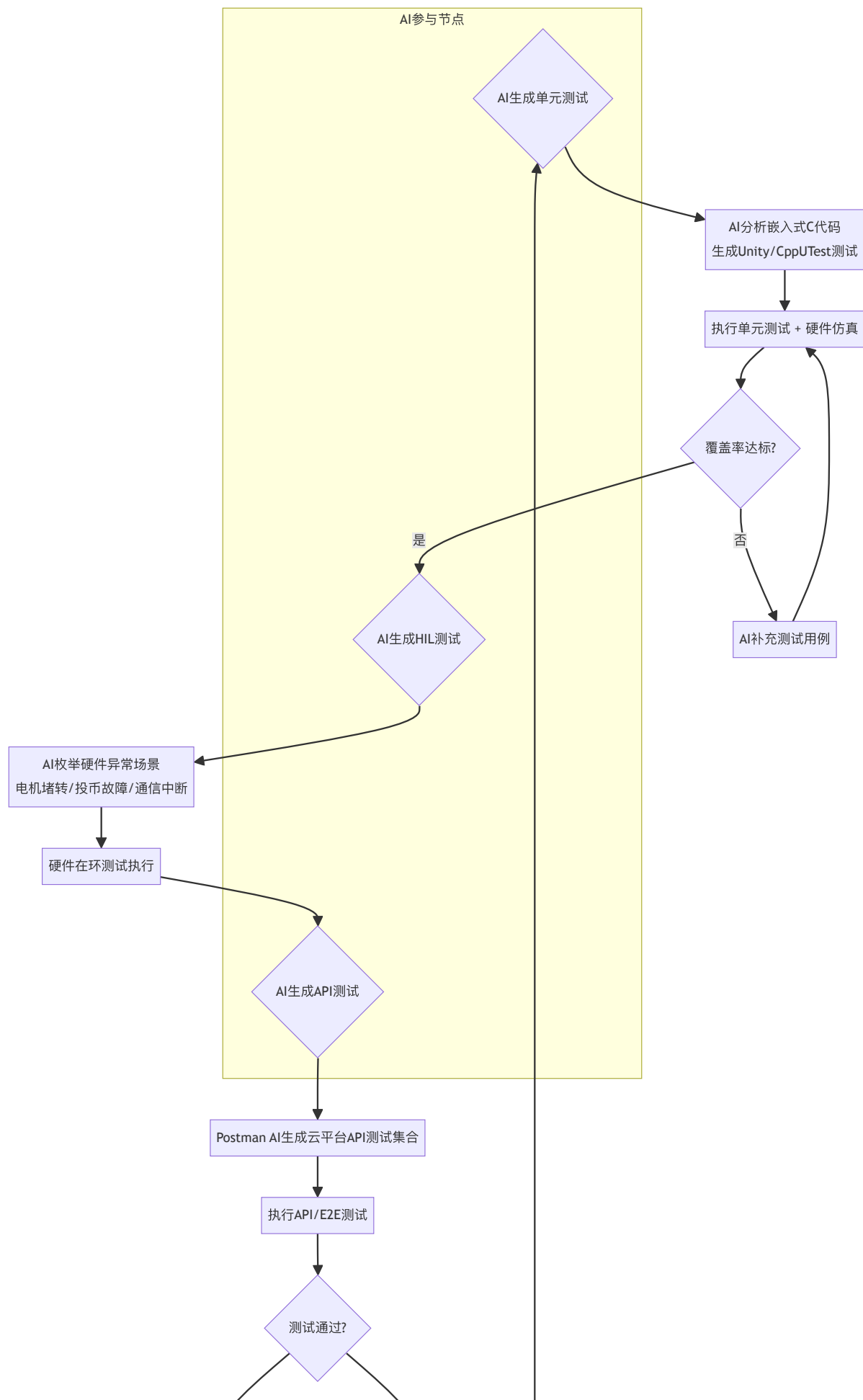


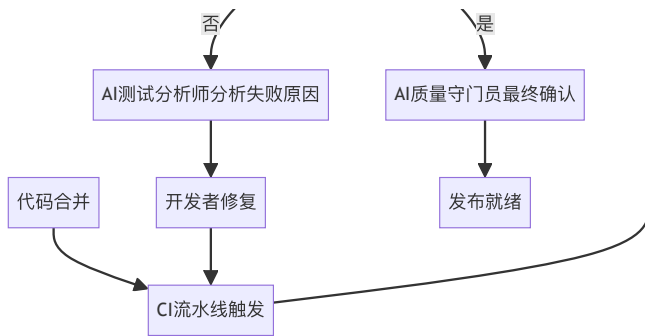
改进点说明

改进项	传统方式	AI增强方式	预期效果
嵌入式C代码	开发者从零编写状态机、驱动代码	AI生成代码骨架、状态机框架和FreeRTOS任务模板	编码速度提升35%
跨栈接口检查	集成阶段手工对齐	AI在编码阶段即自动检查MQTT Topic和REST API的一致性	集成缺陷减少40%
代码审查	全量人工审查（嵌入式代码审查门槛高）	AI先进行首轮安全审查（缓冲区溢出、内存泄漏等），人工聚焦逻辑正确性	审查效率提升55%

3.4 测试验证阶段改进

改进后流程图



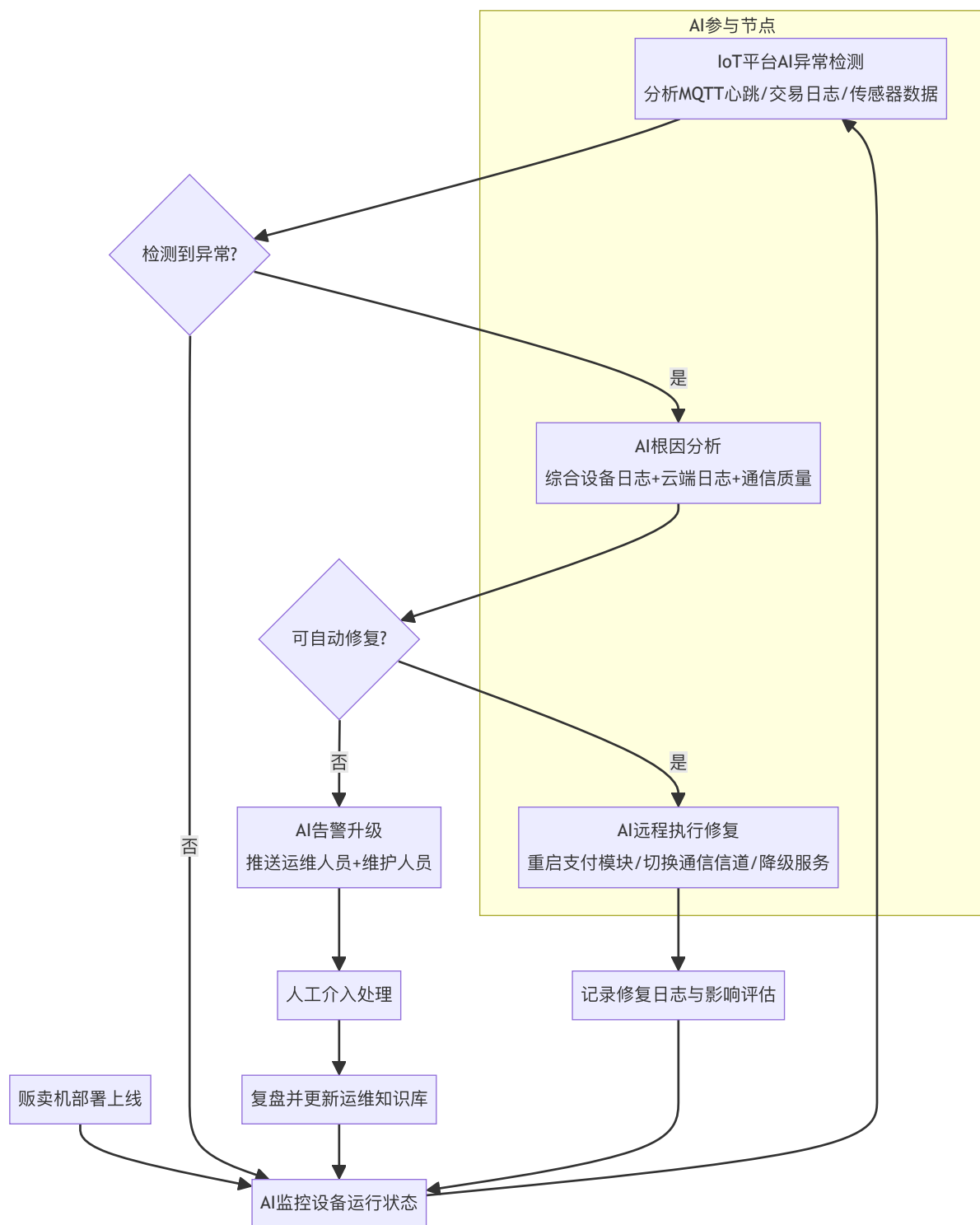


改进点说明

改进项	传统方式	AI增强方式	预期效果
嵌入式单元测试	开发者手工编写（门槛高、耗时）	AI分析C代码自动生成Unity测试框架用例	单元测试编写时间减少60%
HIL测试场景	HIL工程师凭经验枚举故障场景	AI系统性枚举硬件故障×软件状态×通信状态组合	异常场景覆盖率从50%提升至85%
API测试	QA手工编写Postman脚本	AI基于MQTT协议规范和REST API定义自动生成测试集合	API测试生成效率提升65%

3.5 运维监控阶段改进

改进后流程图

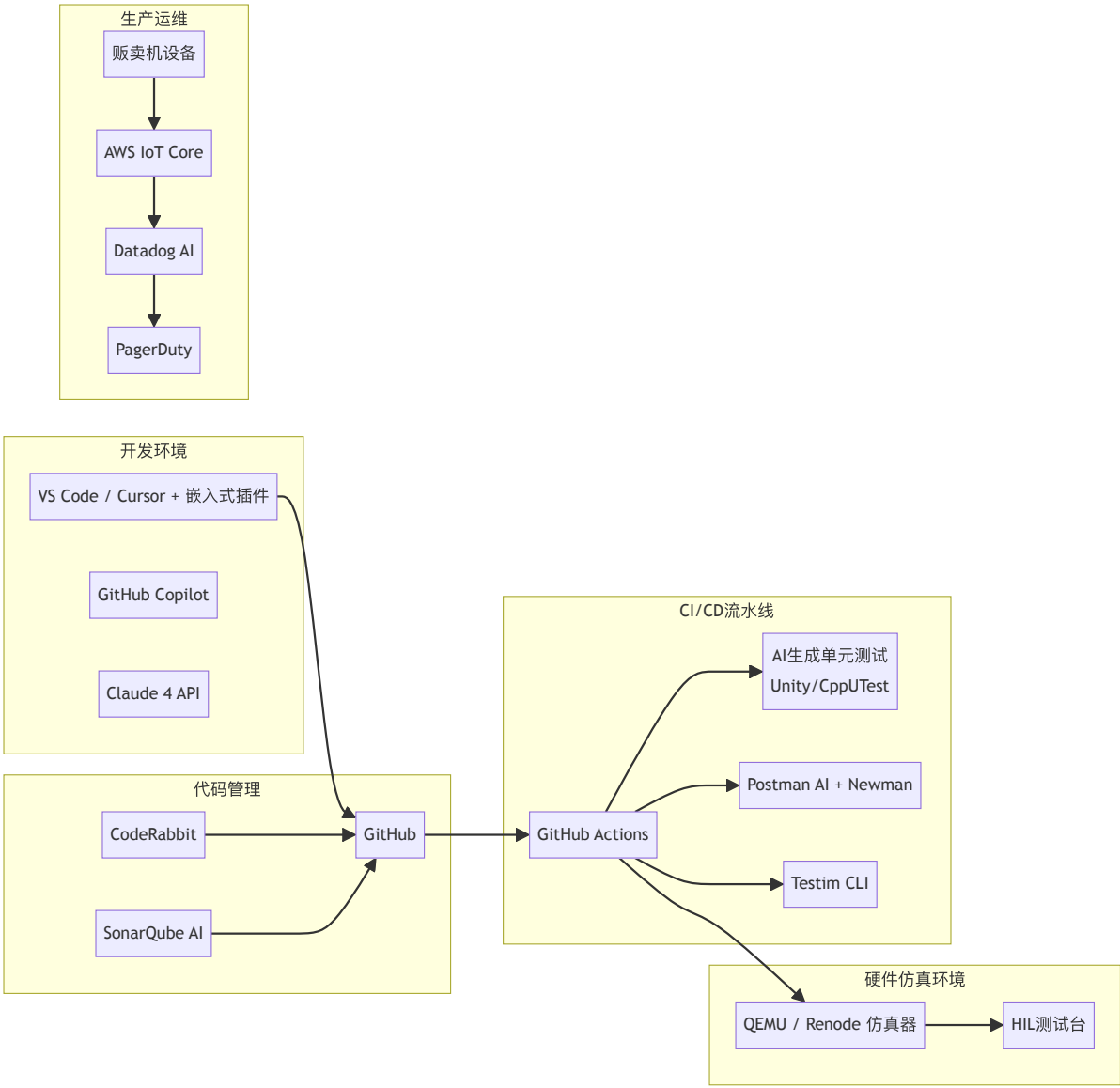


四、工具链选型方案

4.1 完整工具链

SDLC 阶段	AI工具	核心功能	适用子 系统	集成方式	预期效 率提升
需求分 析	Claude 4 / GPT-4o	需求文档生成、一致 性检查、合规检查	全系统	API集成	50%
系统设 计	Claude 4 + Mermaid	架构图、状态机图、 协议规范、ER图生成	全系统	CLI + IDE	50%
嵌入式 编码	GitHub Copilot / 通义 灵码	C/FreeRTOS代码生成 与补全	嵌入式 控制	IDE集成	35%
云平台 编码	Cursor / GitHub Copilot	Java/Python代码生 成	云平台	IDE集成	45%
移动端 编码	Cursor / GitHub Copilot	Vue.js代码生成	移动端	IDE集成	45%
代码审 查	CodeRabbit + SonarQube AI	C代码安全审查、全 栈代码质量	全系统	GitHub App + CI	55%
嵌入式 测试	Claude 4 + Unity/CppUTest	嵌入式单元测试、HIL 场景枚举	嵌入式	CLI + 仿真 平台	60%
API测 试	Postman AI Assistant	REST API & MQTT协 议测试	云平台 +通信 层	Postman 内置	65%
E2E测 试	Testim	移动端和Web管理后 台E2E测试	移动端 +管理 后台	CI集成	60%
运维监 控	AWS IoT Device Defender + Datadog AI	IoT设备异常检测、根 因分析	全系统	Agent + 云 服务	40%

4.2 工具链集成架构



五、AI输出物质量标准与验证机制

5.1 质量标准定义

AI输出物类型	质量标准	特殊要求（嵌入式场景）	验证机制	责任人
AI生成的需求文档	歧义率 < 5%	实时性约束必须量化（如"响应时间 < 2s"）	人工评审 + AI一致性检查	产品经理
AI生成的状态机设计	状态覆盖率 100%，无死锁状态	状态转换须考虑所有硬件中断场景	架构师形式化验证	系统架构师

AI输出物类型	质量标准	特殊要求（嵌入式场景）	验证机制	责任人
AI生成的嵌入式C代码	无缓冲区溢出、无内存泄漏、MISRA-C符合率 > 90%	中断服务程序（ISR）代码须人工逐行审查	SonarQube + CodeRabbit	AI质量守门员
AI生成的HIL测试用例	异常场景覆盖率 > 85%	生成的硬件故障场景须经HIL工程师确认物理可行性	HIL工程师审核	AI测试分析师
AI生成的协议规范	嵌入式与云端定义100%一致	Topic命名、Payload格式、QoS级别须三方一致	AI自动交叉检查	AI质量守门员

5.2 AI输出追溯要求

- 所有AI生成的产出物必须标注：
- AI工具名称与版本
 - 使用的Prompt模板ID
 - 生成时间戳
 - 适用子系统标记（嵌入式/云平台/移动端/全系统）
 - 人工审核状态（待审核/已通过/已驳回）
 - 审核人与审核时间

六、风险评估与应对策略

6.1 风险矩阵

风险编号	风险描述	影响领域	概率	影响	风险等级
R1	数据安全风险： 嵌入式源码上传云端AI工具导致固件安全机制泄露	嵌入式编码	中	高	🔴 高
R2	代码安全风险： AI生成嵌入式C代码存在隐蔽的内存安全问题	嵌入式编码	中	极高	🔴 极高

风险编号	风险描述	影响领域	概率	影响	风险等级
R3	实时性风险： AI不理解实时约束，生成可能破坏实时性的代码	嵌入式编码	高	高	🔴 高
R4	硬件兼容性风险： AI对MCU外设和硬件接口不了解，生成不可用代码	编码、测试	高	中	🟡 中
R5	模型偏见风险： AI生成的测试偏向特定场景，忽略嵌入式特有的边界条件	测试	中	中	🟡 中
R6	技术依赖风险： 嵌入式开发团队过度依赖AI，核心能力退化	全流程	低	高	🟡 中
R7	HIL测试误报： AI生成的HIL场景物理不可行导致无效测试	测试	中	中	🟡 中
R8	支付安全风险： AI辅助代码引入支付流程的逻辑缺陷	嵌入式、云平台	低	极高	🔴 高

6.2 应对策略

R1 — 数据安全风险

- 措施：优先选择支持私有化部署的AI工具（通义灵码企业版）
- 措施：制定《贩卖机项目AI工具数据安全使用规范》，禁止将密钥、支付证书、固件签名私钥输入AI工具
- 措施：代码上传前自动扫描敏感信息（正则匹配密钥格式）

R2 — 代码安全风险（嵌入式最高优先）

- 措施：AI生成的嵌入式C代码**强制**通过SonarQube + CodeRabbit双重安全审查
- 措施：涉及内存操作（malloc/free）、指针运算、中断服务程序（ISR）的AI代码须人工逐行审查
- 措施：建立嵌入式C代码AI使用的"白名单/黑名单"规则——ISR代码禁止AI生成，状态机代码允许AI辅助

R3 — 实时性风险

- 措施：AI生成的涉及实时约束的代码必须附带WCET（最坏执行时间）分析
- 措施：在AI System Prompt中明确标注"本项目为FreeRTOS实时系统，所有任务须考虑优先级和阻塞时间"

R4 — 硬件兼容性风险

- 措施：将MCU数据手册、外设寄存器映射、GPIO引脚定义纳入AI上下文知识库

- 措施：AI生成的外设驱动代码必须通过硬件仿真器（QEMU/Renode）验证

R5 — 模型偏见风险

- 措施：AI测试分析师定期抽样评估AI生成测试的多样性和覆盖率
- 措施：维护嵌入式特有的"边界条件检查清单"（电源波动、EMC干扰、温度范围等）

R6 — 技术依赖风险

- 措施：保留无AI的"裸机开发日"（每月2天），验证团队核心嵌入式开发能力
- 措施：Code Review中特别标注AI生成代码段，审查者须回答"你能独立实现吗？"

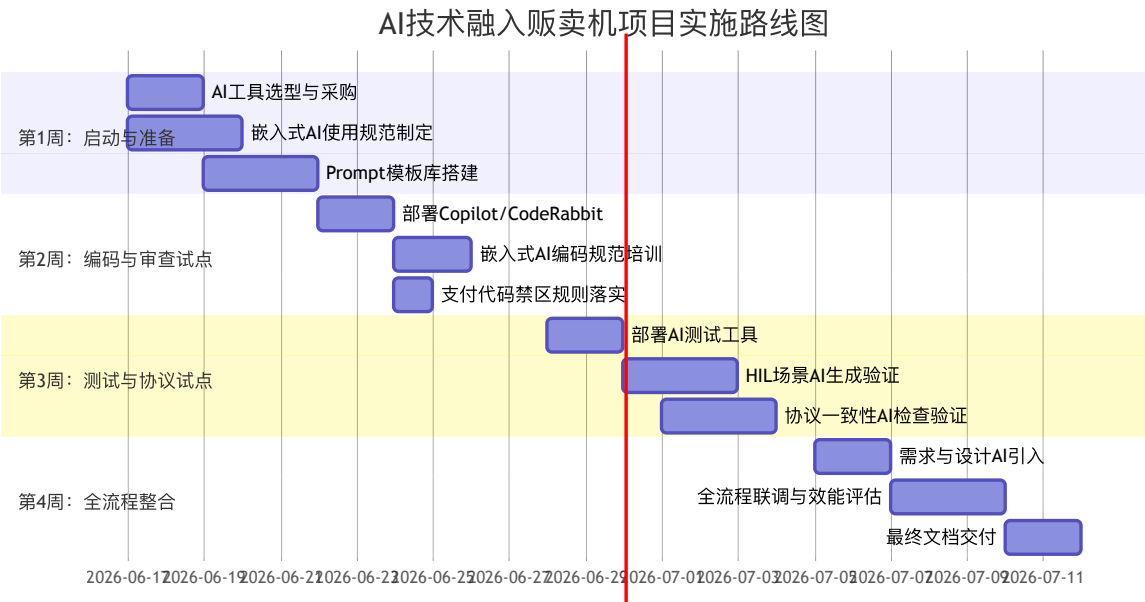
R7 — HIL测试误报

- 措施：AI生成的HIL异常场景须经HIL工程师审核物理可行性后方可进入测试执行
- 措施：建立"已验证物理可行的HIL场景库"，AI基于库中场景进行扩展而非凭空生成

R8 — 支付安全风险

- 措施：支付相关代码（嵌入式支付模块 + 云端支付网关）禁止使用AI生成，须100%人工编写和审查
- 措施：支付代码变更须通过独立的PCI-DSS安全审计

七、实施路线图



八、参考文献

- [1] ISO/IEC/IEEE 12207:2017, "Systems and software engineering — Software life cycle processes", 2017.
- [2] ISO/IEC 33001:2015, "Information technology — Process assessment — Concepts and terminology", 2015.
- [3] 中国信通院, 《智能化软件工程技术和应用要求 第3部分: 智能测试能力》, 2024.
- [4] GitHub, "GitHub Copilot 2025 Impact Report", 2025.
- [5] Gartner, "Predicts 2025: AI in Software Engineering", 2025.
- [6] AWS, "AWS IoT Device Defender 2025 — ML-based Anomaly Detection for IoT Fleets", 2025.